Project B. Number Plate Recognition by Group 1

Suber Abdi w2069210

Christian Dave Bergonia w1985516

Caleb Suleaudu w1910808

Table of Contents

Introduction

In this project the aim is to design a system with its own GUI that can take three images of licence plates from a dataset and isolate and identify them. This system uses image processing techniques such as greyscale conversion, thresholding, noise removal, morphological operations, binarization to enhance the images so that is easier for the image to be read and the number plate to be isolate from the vehicle image. This system is made in MATLAB which lets us use these techniques that can make the number plates easier to identify as they help improve the quality of the image.

1. Theory and Methods

1.1 Greyscale Conversion

Greyscale conversion is the process of transforming a colour image into a single channel representation where each pixel value represents only intensity information. An RGB colour image is an M×N×3 array of colour pixels, where each pixel corresponds to red, green and blue components (Reni, 2025). Converting colour images to greyscale enables faster processing and reduces computational complexity, which is particularly beneficial when performing subsequent operations such as morphological processing (Reni, 2025). The conversion process involves transforming a 24-bit colour value to an 8-bit grey value, where the greyscale of a colour image represents the luminance of the pixel value ranging from 0 to 255 (Reni, 2025).

Choosing an appropriate greyscale conversion technique is crucial, as any loss of information during the conversion from one colour space to another can impair the accuracy of subsequent processing results (Reni, 2025). Different methods are commonly employed for greyscale conversion, including simple averaging ((R+G+B)/3), maximum and minimum averaging ([Max (R, G, B) + Min (R, G, B)]/2), and weighted averaging where R, G and B values are combined with chosen weights (Reni, 2025). The weighted average conversion method is the best approach since it preserves more feature details than other conversion techniques (Reni, 2025). This method is based on the sensitivity of the human eye and uses optimised weights for each colour component (Reni, 2025).

The formula used for greyscale conversion is given by: Grey = W_R R + W_G G + W_B B, where W_R, W_G and W_B are the optimum weights of the red (R), green (G) and blue (B) component values of the image respectively (Reni, 2025). The weights of all three channels must sum to 1 (Reni, 2025). The standard greyscale conversion, established by the National Television Standards Committee (NTSC), uses a weighted sum of R, G and B components with W_R = 0.299, W_G = 0.587 and W_B = 0.114 (Reni, 2025), (Güneş, A et al., 2016). These coefficients reflect the human eye's varying sensitivity to different wavelengths, with greater sensitivity to green (Güneş, A et al., 2016). The luminance obtained through this

weighted combination provides intensity information whilst considering the unequal spectral power contributions of each colour channel (Güneş, A et al., 2016).

1.2 Thresholding

Thresholding is a technique that selects pixels with intensity values and processes them accordingly (Reni, 2025). It is a key process used in image binarization, where based on a threshold value, pixel intensities greater than the threshold are set to 1 and those less than the threshold are set to 0 (Reni, 2025). This technique can be used to identify objects in an image if their intensity values are known and is widely employed in segmentation algorithms as well as pattern recognition algorithms (Reni, 2025). The operations range from uniform thresholding, which applies a single threshold value across the entire image, to adaptive thresholding, which calculates different threshold values for different regions of the image (Reni, 2025).

In medical imaging applications, thresholding-based segmentation approaches have proven particularly useful for dividing images into regions that are homogeneous with respect to specific characteristics (Sandra, J et al., 2023). The main goal of the segmentation process is to divide an image into parts that are correlated with normal anatomy or lesions, simplifying its representation into something more meaningful and easier to analyse (Sandra, J et al., 2023). Thresholding methods address challenges such as regions with missing edges, lack of texture contrast, and distinguishing between the region of interest and background (Sandra, J et al., 2023). These approaches have been proposed with promising results for creating systems that assist in the process of diagnosis and medical decision making across various medical imaging modalities including computed tomography, magnetic resonance imaging, ultrasound and X ray (Sandra, J et al., 2023).

1.3 Noise Removal

Digital images are prone to various types of noise that are formed because of errors in the image acquisition process, making noise removal techniques an essential preprocessing practice in imaging (Reni, 2025). Noise is a random variation of image intensity that is visible as grains in the image (Verma, R et al., 2013). It may arise as effects of basic physics, such as the photon nature of light or thermal energy of heat inside the image sensors and may be produced at the time of capturing or image transmission (Verma, R et al., 2013). Noises are induced by the image sensors and scanner or digital camera circuit (Reni, 2025). When noise is present, the pixels in the image show different intensity values instead of true pixel values (Verma, R et al., 2013).

The most commonly occurring types of noise include salt and pepper noise, which is random impulse noise with sharp and sudden disturbances in the image signal caused due to analogue to digital converter errors (Reni, 2025). Gaussian noise is a statistical noise caused by the sensors which degrades image quality, where the sensor has inherent noise due to the level of illumination and its own temperature, and the electronic circuits connected to

the sensor inject their own share of electronic circuit noise (Reni, 2025). Poisson noise, also known as shot noise, is a type of electronic noise which originates from the discrete nature of electric charge and is due to the variation in the number of photons sensed at a given exposure level (Reni, 2025). Grain noise occurs if the image is scanned from a photograph made on film, where the film grain is a source of noise, and noise can also be the result of damage to the film or be introduced by the scanner itself (Reni, 2025).

Noise removal algorithms are processes of removing or reducing the noise from the image (Verma, R et al., 2013). These algorithms reduce or remove the visibility of noise by smoothing the entire image whilst leaving areas near contrast boundaries, though these methods can obscure fine, low contrast details (Verma, R et al., 2013). Different filtering techniques are employed depending on the type of noise present. Spatial filters such as median and mean filters perform better for removing salt and pepper noise and grain noise for images in greyscale (Reni, 2025). Wiener filter performs better for removing Poisson and Gaussian noise (Reni, 2025). The selection of appropriate filtering techniques is crucial for effective noise reduction whilst preserving important image features.

1.4 Morphological Operations

Morphological operations are mathematical techniques typically performed on binary images, though the concepts can be extended to greyscale images (Reni, 2025). These operations serve multiple critical purposes in image processing: removing imperfections such as noise and texture from binary images created by segmentation techniques, identifying form and structure present in an image, and performing non-linear smoothing and feature enhancement (Reni, 2025). Morphology fundamentally deals with notions of shape, structure and size within digital images (Reni, 2025). The core principle involves adding or removing pixels at the periphery of objects according to specific rules using a structuring element (SE), which for binary images is typically represented as a small matrix of binary-valued pixels (Reni, 2025). The shape of the structuring element directly influences the operation's results, with differently shaped SEs producing different morphological effects (Reni, 2025).

The fundamental morphological operations are erosion and dilation. Erosion shrinks foreground regions by removing pixels at object boundaries, removing small protrusions, breaking thin connections between objects, and eliminating isolated foreground pixels smaller than the structuring element. Dilation expands foreground regions by adding pixels to object boundaries, filling small holes within objects, connecting nearby regions, and thickening existing structures. Opening combines erosion followed by dilation using the same structuring element, removing small objects and thin protrusions while preserving the overall size and shape of larger objects. Closing performs dilation followed by erosion, filling small gaps and holes while maintaining object boundaries. These composite operations provide more sophisticated control over image structure than erosion or dilation alone.

Morphological operations prove extremely useful in isolating specific regions of interest within an image for subsequent processing stages (Reni, 2025). They facilitate the removal or addition of certain types of pixels within defined neighbourhoods and are employed to extract the skeleton of objects present in images (Reni, 2025). In medical imaging applications such as DNA microarray image processing, morphological operations using basic erosion and dilation can enhance image quality by differentiating between foreground and background structures (K. A. M. Said et al., 2016). The performance of morphological operations depends critically on the characteristics of the chosen structuring element (K. A. M. Said et al., 2016).

1.5 Binarization

Binarization is the process of converting colour or greyscale images into binary images, which are arrays of 0s and 1s represented as logical arrays (Reni, 2025). A binary image is only black and white without intermediate greys (Reni, 2025). This process is achieved through thresholding, where based on a threshold value, pixel intensities greater than the threshold are set to 1 and those less than the threshold are set to 0 (Reni, 2025). Binary image representation, a black and white representation of object and non-object (background), is the preferred format for image analysis, especially document image analysis which consists of textual objects (Ismail, S et al., 2018).

In document image analysis applications, binarization is a crucial preprocessing step whose accuracy affects the performance of subsequent stages in detecting and extracting text from document images (Ismail, S et al., 2018). The purposes of binarization include noise removal and size reduction of images in memory (Ismail, S et al., 2018). This process increases the visibility of important information in an image by discarding unimportant information, preserving important image information by reducing all levels in a colour or greyscale image into two levels: black text and white background (Ismail, S et al., 2018). However, thresholding only considers pixel intensity and disregards relationships between pixels, which is a major limitation whereby noise pixels can be mistakenly included and informative pixels isolated from others can be missed (Ismail, S et al., 2018).

2. Project Methodology

2.0 System Overview

For this project MATLAB R2025b was used to design a graphic user interface (GUI), which will allow its user to upload three separate images, gives the values of the contrast of the images, convert the images to greyscale and display, display histograms of the images. Then after use the appropriate pre-processing techniques and filtering operations to remove noise and enhance the images.

2.1 GUI

To make the GUI that the program needed the properties of the components of the app need to be declared (Figure 2). These components that have just be declared need the information for their purpose on the GUI (Figure 3) such as type, position, colour, label and if it is connected to a button/function.

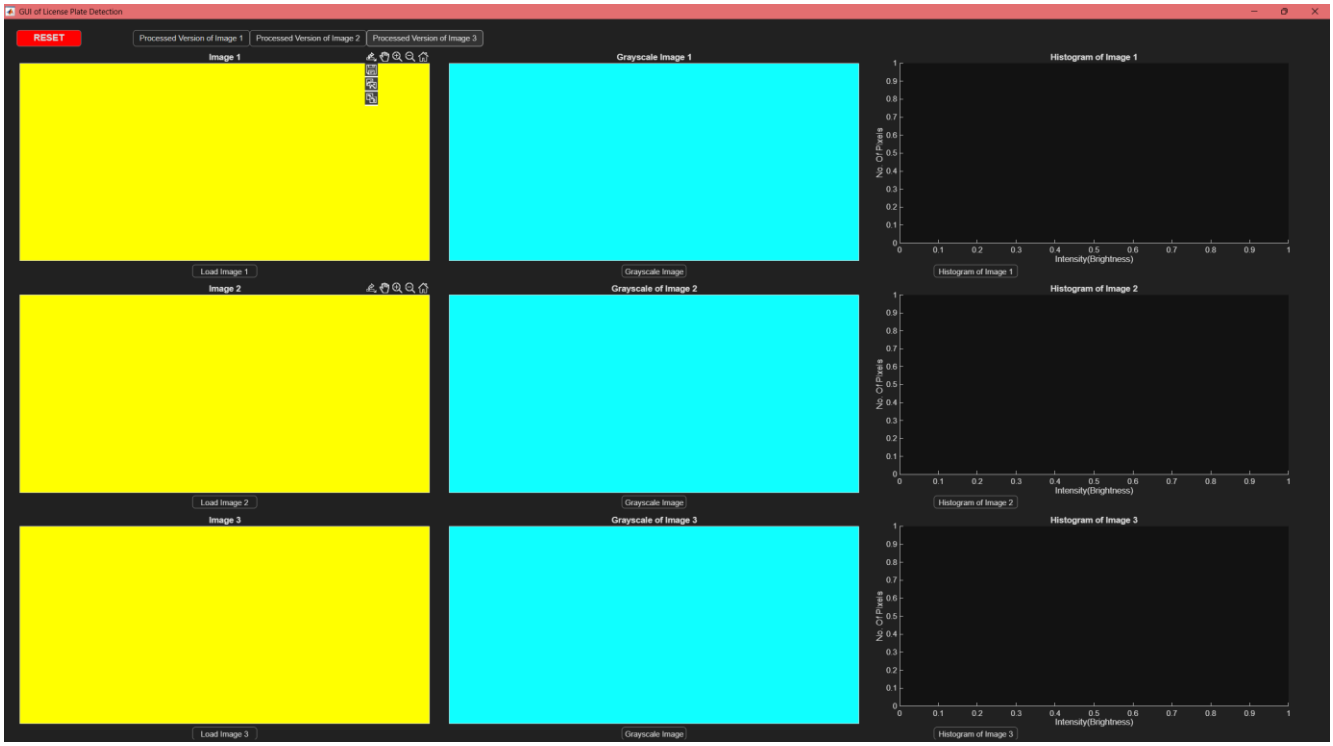

Figure 1: The GUI of the program

```
properties (Access = public)
    UIFigure matlab.ui.Figure
    HistogramButton matlab.ui.control.Button
    Histogram2Button matlab.ui.control.Button
    Histogram3Button matlab.ui.control.Button
    RESETButton matlab.ui.control.Button
    GrayscaleImageButton matlab.ui.control.Button
    GrayscaleImage2Button matlab.ui.control.Button
    GrayscaleImage3Button matlab.ui.control.Button
    LoadImageButton matlab.ui.control.Button
    LoadImage2Button matlab.ui.control.Button
    LoadImage3Button matlab.ui.control.Button
    ProcessedImageButton matlab.ui.control.Button
    ProcessedImage2Button matlab.ui.control.Button
    ProcessedImage3Button matlab.ui.control.Button
    UIAxes9 matlab.ui.control.UIAxes
    UIAxes8 matlab.ui.control.UIAxes
    UIAxes7 matlab.ui.control.UIAxes
    UIAxes6 matlab.ui.control.UIAxes
    UIAxes5 matlab.ui.control.UIAxes
    UIAxes4 matlab.ui.control.UIAxes
    UIAxes3 matlab.ui.control.UIAxes
    UIAxes2 matlab.ui.control.UIAxes
    UIAxes matlab.ui.control.UIAxes
end
```

Figure 2: Properties of app components

```
% Component initialization
methods (Access = private)

    % Create UIFigure and components
    function createComponents(app)

        % Create UIFigure and hide until all components are created
        app.UIFigure = uifigure('Visible', 'off');
        app.UIFigure.Position = [100 100 960 600];
        app.UIFigure.Name = 'GUI of License Plate Detection';

        % Create UIAxes
        app.UIAxes = uiaxes(app.UIFigure);
        title(app.UIAxes, 'Image 1')
        app.UIAxes.XTick = [];
        app.UIAxes.YTick = [];
        app.UIAxes.Color = [1 1 0];
        app.UIAxes.Box = 'on';
        app.UIAxes.ButtonDownFcn = createCallbackFcn(app, @UIAxesButtonDown, true);
        colormap(app.UIAxes, 'summer')
        app.UIAxes.Position = [20 400 280 150];

        % Create UIAxes2
        app.UIAxes2 = uiaxes(app.UIFigure);
        title(app.UIAxes2, 'Grayscale Image 1')
        app.UIAxes2.XTick = [];
        app.UIAxes2.YTick = [];
        app.UIAxes2.Color = [0.0588 1 1];
        app.UIAxes2.Box = 'on';
        app.UIAxes2.Position = [320 400 280 150];

        % Create UIAxes3
        app.UIAxes3 = uiaxes(app.UIFigure);
        title(app.UIAxes3, 'Histogram of Image 1')
        xlabel(app.UIAxes3, 'Intensity(Brightness)')
        ylabel(app.UIAxes3, 'No. Of Pixels')
        zlabel(app.UIAxes3, 'Z')
        app.UIAxes3.Position = [620 400 280 150];
```

Figure 3: All the variables and details of each part of the GUI

```
    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = Final

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```

Figure 4: The function where the app (GUI) is generated

2.2 Upload and Display

```
% Button pushed function: LoadImageButton
function LoadImageButtonPushed(app, ~)
    global I;
    [filename,pathname]= uigetfile('*.*', 'Load Image'); % opens file explorer so that an image can be selected
    filename=strcat(pathname,filename); % gets and saves the pathway of the image
    I=imread(filename); % saves the image to the variable
    imshow(I,'Parent',app.UIAxes); % displays the image on GUI
end
```

Figure 5: Code from the program that will allow for an image to be uploaded

The code in Figure 5 shows a function that will allow the user to upload an image from the files of their computer. The image that is selected will be attached to the variable I (the image); the last line of the function will send this image to the be used in the GUI.

Figure 6: The setup of the button in the properties of the MATLAB files (app components)

```
% Create LoadImageButton
app.LoadImageButton = uibutton(app.UIFigure, 'push'); % connecting the button to the GUI
app.LoadImageButton.ButtonPushedFcn = createCallbackFcn(app, @LoadImageButtonPushed, true); % causes the function to run when the button is pressed
app.LoadImageButton.Position = [110 380 100 20]; % where the button is located on the GUI
app.LoadImageButton.Text = 'Load Image 1'; % the label attached to the button
```

Figure 7: The creation of the loading button for the GUI

In Figure 7, this part of the code sets up the button, it has been made it so that in second line of code when that button is pressed then the function will occur, where the position of the button will be on the GUI and what text the button will have labelled on it. The two figures (8 & 9) show what the GUI looks like before after using these functions.

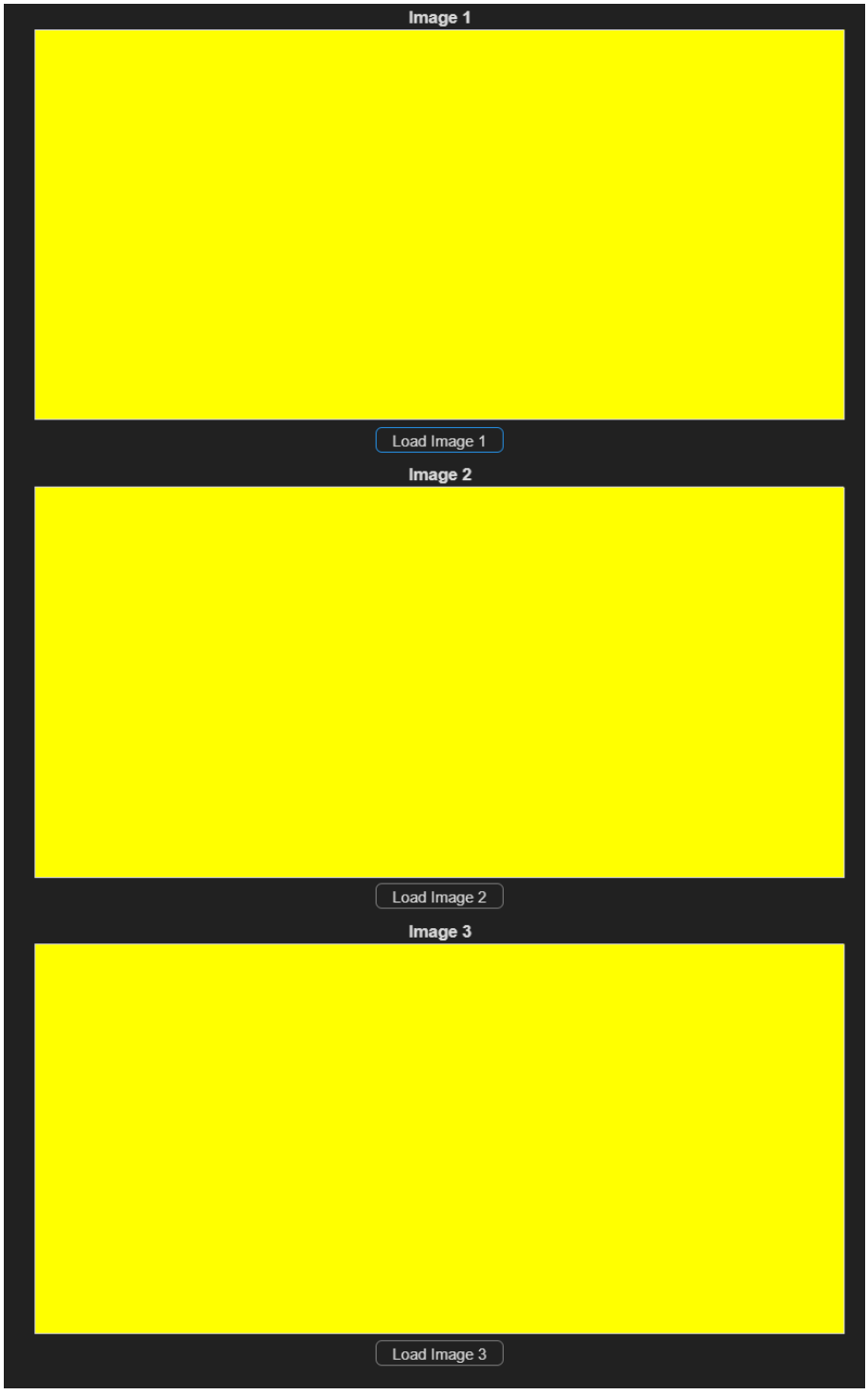

Figure 8: GUI before loading the image

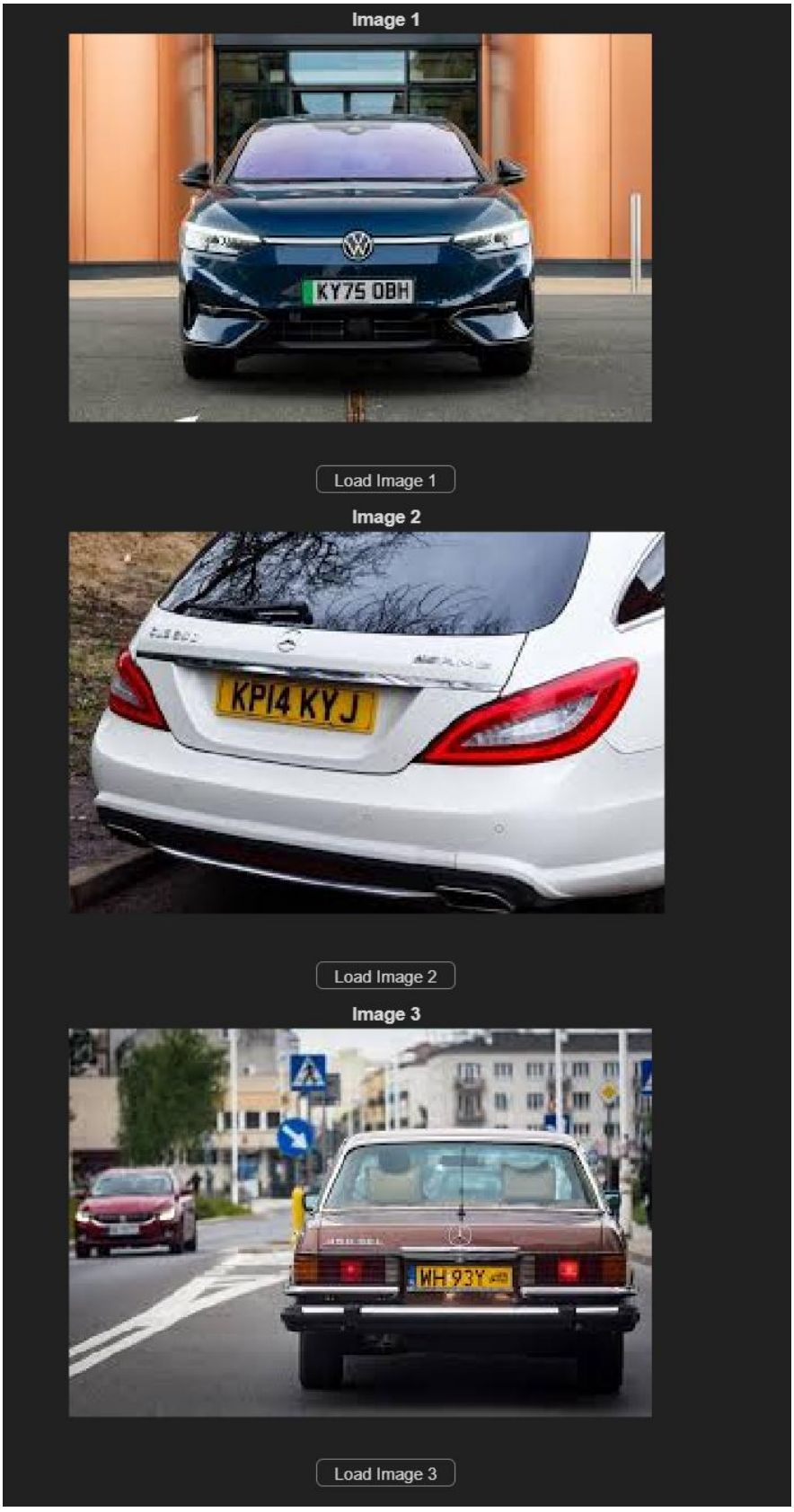

Figure 9: GUI after selecting the image

2.3 RMS Contrast

In Figure 10, it shows how this function was used to work out the contrast of the image that was uploaded. The first line gets the size of the image (number of rows and columns), the second line gets the mean of the brightness of the image, the third line use all these values in an equation to work out the standard deviation of the image. When the program is run and a button is pressed to convert an image to grayscale it will output the value of the contrast of the image, the code where the function is run and results of this can be seen in Figure 11 and 12.

```
function C = RMSContrast(I) %function to work out contrast of images
    [M, N] = size(I); % size of the image (no. of rows and columns)
    me = mean(I(:)); % average brightness of image
    C = sqrt(sum((I(:) - me).^2)/(M*N)); % RMS Equation
end
```

Figure 10: The function that uses the RMS Contrast formula to find out the contrast of the image

```
function GrayscaleImageButtonPushed(app, ~)
    global I;
    global J;
    C = RMSContrast(I); % Running this function, saving is value to C
    fprintf('The value of RMS is %f\n', C); % Outputting the contrast of this image
```

Figure 11: Code which runs the RMS Contrast Function

```
ans =

  Final with properties:

                UIFigure: [1×1 Figure]
        HistogramButton: [1×1 Button]
       Histogram2Button: [1×1 Button]
       Histogram3Button: [1×1 Button]
            RESETButton: [1×1 Button]
    GrayscaleImageButton: [1×1 Button]
   GrayscaleImage2Button: [1×1 Button]
   GrayscaleImage3Button: [1×1 Button]
         LoadImageButton: [1×1 Button]
        LoadImage2Button: [1×1 Button]
        LoadImage3Button: [1×1 Button]
    ProcessedImageButton: [1×1 Button]
   ProcessedImage2Button: [1×1 Button]
   ProcessedImage3Button: [1×1 Button]
                UIAxes9: [1×1 UIAxes]
                UIAxes8: [1×1 UIAxes]
                UIAxes7: [1×1 UIAxes]
                UIAxes6: [1×1 UIAxes]
                UIAxes5: [1×1 UIAxes]
                UIAxes4: [1×1 UIAxes]
                UIAxes3: [1×1 UIAxes]
                UIAxes2: [1×1 UIAxes]
                 UIAxes: [1×1 UIAxes]

The value of RMS is 10.704410
```

Figure 12: The output in the command window of MATLAB when the RMS Contrast function happens

2.4 Grayscale Conversion

In the function in Figure 13, The “extract channels” are to divide the image into red, green and blue whilst the default values of rgb2gray (the built in MATLAB function that converts RGB images to grayscale). Then there is the grayscale formula which should be used to work out what are the best values necessary to convert the RGB image into grayscale.

```
function Gray = Grayscale(I)
    % Extract channels
    R = I(:,:,1);
    G = I(:,:,2);
    B = I(:,:,3);

    % The normal values of rgb2gray
    WR = 0.299;
    WG = 0.587;
    WB = 0.114;

    % Grayscale formula
    Gray = WR*R + WG*G + WB*B;
end
```

Figure 13: function that contain rgb2gray values and will convert an RGB image to grayscale

```
% Button pushed function: GrayscaleImageButton
function GrayscaleImageButtonPushed(app, ~)
    global I;
    global J;
    C = RMSContrast(I); % Running this function, saving is value to C
    fprintf('The value of RMS is %f\n', C); % Outputting the contrast of this image
    J = Grayscale(I); % Running this function converts images to grayscale
    imshow(J, 'Parent',app.UIAxes2) % Send the image attached to variable J to the GUI
end
```

Figure 14: Function where the grayscale is runs and converts the image

Figure 15: The grayscale version of all three images

2.5 Histogram

In Figure 16, a histogram has been generated in the GUI by find how many pixels the image has, then for loops will be used to iterate through each pixel. The reason why this function has an empty array of 256 zeros made so that we can update each one. The outer for loop will go through the rows starting with the first, the inner one will do that same but for

column. This means the function will go onto one row then go through every column, after each column is done, it will go to the next row and start again till it reached the last row. Whilst this is happening Var = I (r, c) will check the brightness of the pixel, then H (Var + 1) = H (Var + 1) + 1 will record that variable to whatever level of brightness that pixel is. For example, if a pixel with a brightness of 101 appears then it will add 1 to 101.

As seen in Figure 17, all the histograms vary but will tell us a key detail. The x-axis of the histogram is the intensity(brightness) of the pixel whilst the y-axis is the number of pixels that share the same shade of grey. This means that the if there are more longer lines of the left side the image is closer to black but if there are more longer lines of the right side the image is closer to white.

```
function H = Hist(I)
    [rows, cols] = size(I); % Finds how many pixels the image has
    H = zeros(1, 256); % Creates empty area of 256 zeros

    % For loops to iterate through everything
    for r = 1:rows % Starts at first row
        for c = 1:cols % Starts at first column
            Var = I(r, c);
            H(Var + 1) = H(Var + 1) + 1;
            % The x-axis is the intensity(brightness) of the pixel
            % The y-axis is the no. of pixels that share the same shade of grey
            % If the left side has more lines the image is closer to black,
            % If the right has more the image is closer to white
        end
    end
end
```

Figure 16: The function of the histogram

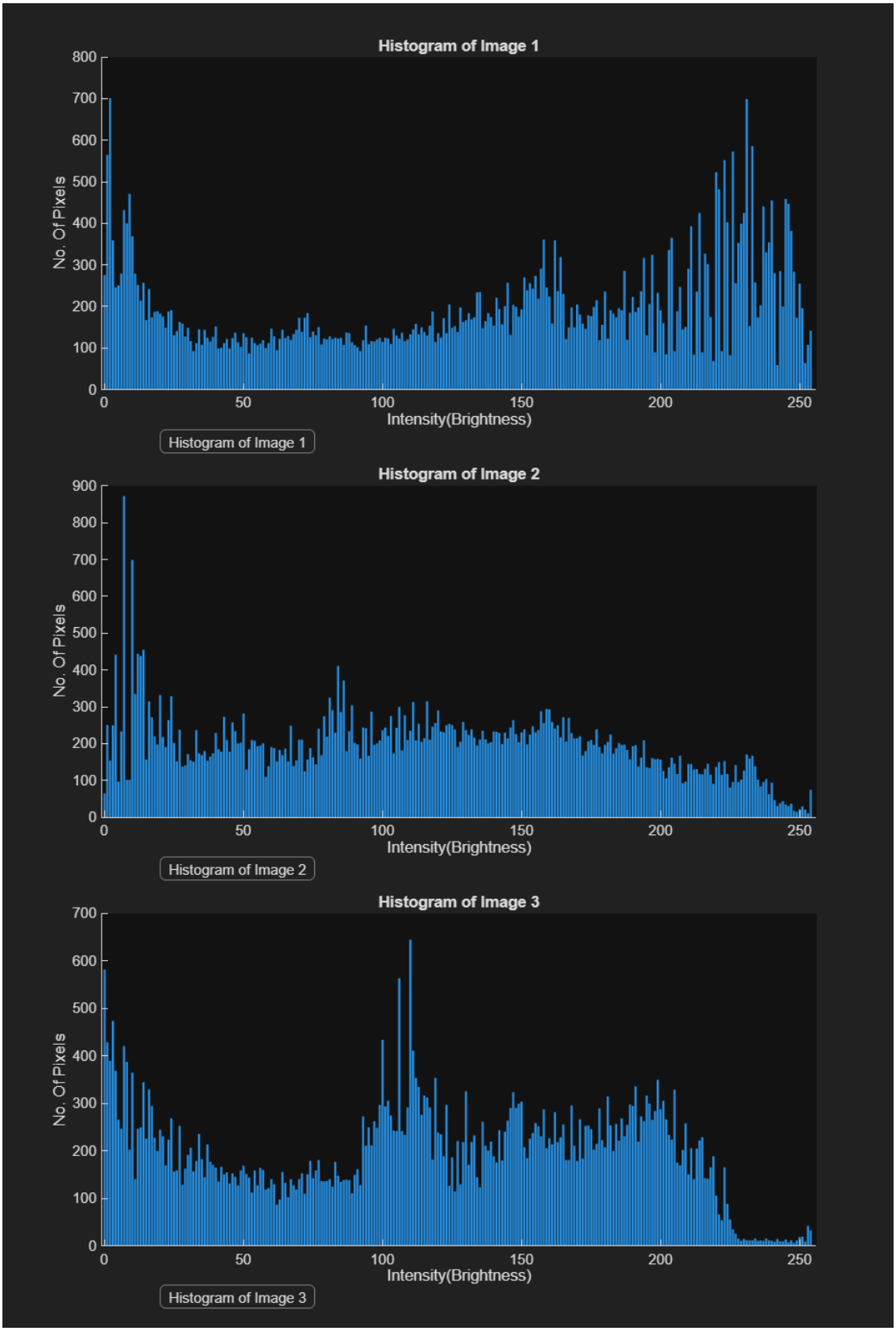

Figure 17: The histograms of the images generated on the GUI

2.6 Pre-Processing Techniques and Filtering Operations

After the images have been loaded into the programming and the images have gone through grayscale, there is a function (Figure 18) in the program that will complete the necessary pre-processing techniques and filtering operations so that making it easier to identify number plates.

```
function MO = Processing(J)
    J=medfilt2(J,[3,3]); % Median filter
    SE = strel('rectangle', [3 15]); % Structuring element
    b = imbinarize(J); % Binarization
    IO = imopen(b,SE); % Opening
    MO = imclose (IO,SE); % Closing
end
```

Figure 18: The function with all the pre-processing techniques and filtering operations

The first line of code in the function is a median filter which will help get rid of “salt and pepper” noise whilst keeping the sharp edges of the license plate and not blurring them, this is why a median filter is used instead of a gaussian filter (Reni, 2025). The next line of code creates a structuring element which used to preserve part of the images that are similar to the structuring element (Reni, 2025). This is why the structuring element is a rectangle, and its dimension are 3x15 which is similar to the average licence plate (Reni, 2025).

Afterwards, binarization occurred so that the image is converted to a binary image just 0 (Black) and 1 (White) (Reni, 2025). This is so that irrelevant details are removed for the processed image needed and so that morphological operations can be used (Reni, 2025). The last part of this function are the morphological operations, where “opening” (erosion followed by dilation) happens first to remove noise without affecting important details (Reni, 2025). Then “closing” (dilation followed by erosion) happens so that small hole and gaps are filled in between the nearby objects, this is to help creating a clear rectangular shape of the license plate (Reni, 2025). The result can be seen in Figure 19; the red circles are where the license plates are.

The issues that come with these techniques is that if the area of the surroundings around the licence plate are similar, they might merge with the license plate, making it impossible to identify. This is why it is harder to make out what part of the image is the license plate is some of the images in Figure 19. In the future it would be best to refine these techniques so that it is easier to separate the license plate from the image.

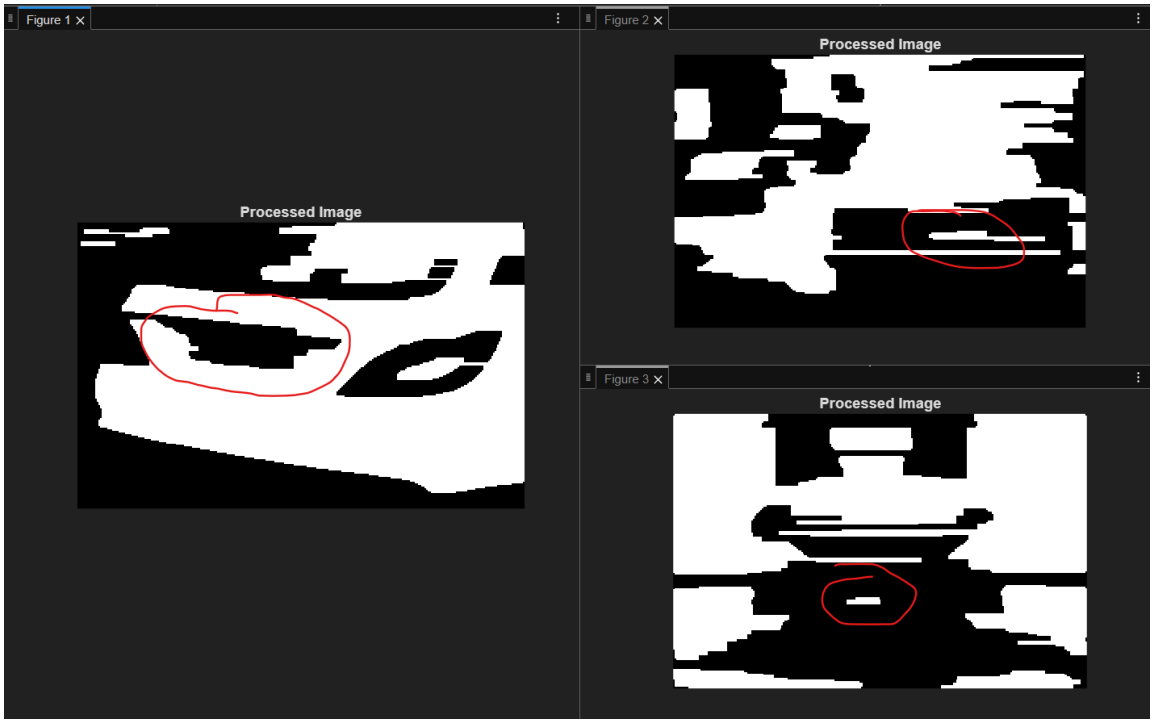

Figure 19: Processed version of images containing license plates

3. Conclusion (Ethical, Societal, Environmental Impact)

3.1 Ethical Concerns

Computer vision is a rapidly evolving field of research that deals with how computers can be made to understand different digital images and videos through analysing, processing and understanding visual content (Lauronen, M., 2017). With any new and topical information technology, questions arise about the ethical practices of its implementation (Lauronen, M., 2017). According to Quinn (2004), there is a need to approach every new technology in a thoughtful manner by thoroughly examining the technology and considering various issues it poses on ethics (Lauronen, M., 2017). The main ethical concerns surrounding computer vision technologies stem from the lack of concrete and cohesive literature that summarises ethical issues of new topical computer vision technologies in one framework (Lauronen, M., 2017). As software applications utilising computer vision tasks become available for public use in different forms of devices and platforms, such as mobile phones and on the Internet, they inevitably pose questions about their ethical implications (Lauronen, M., 2017). In the context of automated number plate recognition systems, ethical concerns include privacy implications of automated vehicle tracking, potential misuse of captured data, surveillance concerns, and the need for transparent data handling practices. These systems must be developed with consideration for professional codes of conduct that prioritise user privacy, data protection, and responsible deployment that respects civil liberties whilst serving legitimate purposes such as traffic management and law enforcement within appropriate legal frameworks.

3.2 Social Impact

Computer vision has developed to become more powerful and present in various fields, impacting the life of everyone in modern society (Antensteiner, D. 2013). The technology is omnipresent across multiple domains: robots help produce the products we use, visual surveillance takes place on our streets, fingerprint sensors are used for security on mobile devices, vision algorithms support medical diagnosis, and industrial inspection tasks are carried out with imaging systems (Antensteiner, D. 2013). Automated number plate recognition systems, such as the one developed in this project, contribute to societal benefits through applications in traffic management, law enforcement, parking management, and toll collection. These systems can enhance public safety by assisting in locating stolen vehicles, identifying traffic violations, and improving urban mobility management. However, they also raise important societal considerations regarding the balance between security benefits and individual privacy rights. The widespread deployment of such systems necessitates clear policies on data retention, access control, and transparency to ensure that technological advancement serves the public interest whilst respecting fundamental rights. Assistive computer vision tools demonstrate the broader positive impact of the technology in overcoming limitations and improving the everyday life of disabled and elderly people, patients, and healthy individuals who benefit from support systems (Antensteiner, D. 2013).

3.3 Environmental Impact

Society is experiencing exponential growth in developing, training, and using deep learning pipelines, particularly within computer vision, with rapid advancement even surpassing human level cognition (Andre, F et al., 2021). However, developing and training state of the art models demand an inordinate computational burden, which corresponds to significant energy and environmental costs (Andre, F et al., 2021). Both the model development phase through architecture search and the lifetime evaluation phase require enormous computational costs and equivalently enormous carbon dioxide emissions (Andre, F et al., 2021). For this number plate recognition project, the environmental impact can be mitigated through several approaches: utilising pretrained models and transfer learning rather than training from scratch, optimising algorithms for computational efficiency, implementing the system on energy efficient hardware, and considering the deployment scale carefully. Whilst individual implementations may have modest environmental footprints, the cumulative impact of widespread deployment necessitates consideration of sustainable computing practices and the balance between performance requirements and environmental responsibility (Andre, F et al., 2021).

3.4 Conclusion

Overall, the program has the ability to convert multiples of three of any images into its grayscale variant and the contrast level of the image. As well as a histogram version of the

image that will state how many pixels of the image contain a certain level (0-255) of brightness (colour of grey). In addition to this there is a preprocessing part that will make it easier for the licence plates to be made out. However, there is still room for improvement as there is not a function that measures and use all possible combination of the RGB values and contrast value of the images so that it is easy to see what the best way is to convert an image to grayscale. Furthermore, it is possible to improve the filtering methods and usage of morphological operations so that it is easier to make the licence plate easier to identify.

References

Saumya Reni. (2025) 'Lecture 07 Morphological Image Processing'. *5ELEN021W.1: Sensors and Signals*. University of Westminster. Unpublished.

Güneş, A., Kalkan, H. & Durmuş, E. Optimizing the color-to-grayscale conversion for image classification. *SIViP* **10**, 853–860 (2016). https://doi.org/10.1007/s11760-015-0828-7

Saumya Reni. (2025) 'Lecture-06 Image Analysis and Operations'. *5ELEN021W.1: Sensors and Signals*. University of Westminster. Unpublished.

Sandra, J., João, A. & Carlos, M. Image thresholding approaches for medical image segmentation - short literature review. (2023). https://doi.org/10.1016/j.procs.2023.01.439

Verma, R. & Jahid, A. (2013) *researchgate*. Available at: https://www.researchgate.net/A-comparative-study-of-various-types-of-image-noise-and-efficient-noise-removal-techniques (Accessed: 01/01/2026).

K. A. M. Said & A. B. Jambek, "A study on image processing using mathematical morphological," 2016 3rd International Conference on Electronic Design (ICED), Phuket, Thailand, 2016, pp. 507-512, https://doi.org/10.1109/ICED.2016.7804697

Ismail, S. M., Sheikh Abdullah, S. N. H. & Fauzi, F. (2018). Statistical Binarization Techniques for Document Image Analysis. *Journal of Computer Science*, *14*(1), 23-36. https://doi.org/10.3844/jcssp.2018.23.36

Lauronen, M. (2017) Ethical issues in topical computer vision applications. Available at: https://jyx.jyu.fi/jyx/Record/Ethical issues in topical computer vision applications (Accessed: 01/01/2026).

Antensteiner, D. (2013) *icvss*. Available at: https://icvss.dmi.unict.it/The Social Impact of Computer Vision (Accessed: 01/01/2026).

Andre Fu, Mahdi S. Hosseini, Konstantinos N. Plataniotis; Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2021, pp. 2311-2317

Appendix

Code

```
classdef Final < matlab.apps.AppBase
% Properties that correspond to app components
properties (Access = public)
UIFigure matlab.ui.Figure
HistogramButton matlab.ui.control.Button
Histogram2Button matlab.ui.control.Button
Histogram3Button matlab.ui.control.Button
RESETButton matlab.ui.control.Button
GrayscaleImageButton matlab.ui.control.Button
GrayscaleImage2Button matlab.ui.control.Button
GrayscaleImage3Button matlab.ui.control.Button
LoadImageButton matlab.ui.control.Button
LoadImage2Button matlab.ui.control.Button
LoadImage3Button matlab.ui.control.Button
ProcessedImageButton matlab.ui.control.Button
ProcessedImage2Button matlab.ui.control.Button
ProcessedImage3Button matlab.ui.control.Button
UIAxes9 matlab.ui.control.UIAxes
UIAxes8 matlab.ui.control.UIAxes
UIAxes7 matlab.ui.control.UIAxes
UIAxes6 matlab.ui.control.UIAxes
UIAxes5 matlab.ui.control.UIAxes
UIAxes4 matlab.ui.control.UIAxes
UIAxes3 matlab.ui.control.UIAxes
UIAxes2 matlab.ui.control.UIAxes
UIAxes matlab.ui.control.UIAxes
end
% Callbacks that handle component events
methods (Access = private)
% Button down function: UIAxes
function UIAxesButtonDown(~, ~)
end
% Button pushed function: LoadImageButton
function LoadImageButtonPushed(app, ~)
global I;
[filename,pathname]= uigetfile('*.*', 'Load Image'); % opens file explorer so that
an image can be selected
filename=strcat(pathname,filename); % gets and saves the pathway of the image
I=imread(filename); % saves the image to the variable
imshow(I,'Parent',app.UIAxes); % displays the image on GUI
end
% Button pushed function: LoadImage2Button
function LoadImage2ButtonPushed(app, ~)
global I2;
[filename,pathname]= uigetfile('*.*', 'Load Image');
filename=strcat(pathname,filename);
```

```
I2=imread(filename);
imshow(I2,'Parent',app.UIAxes4);
end
% Button pushed function: LoadImage3Button
function LoadImage3ButtonPushed(app, ~)
global I3;
[filename,pathname]= uigetfile('*.*', 'Load Image');
filename=strcat(pathname,filename);
I3=imread(filename);
imshow(I3,'Parent',app.UIAxes7);
end
% Button pushed function: GrayscaleImageButton
function GrayscaleImageButtonPushed(app, ~)
global I;
global J;
C = RMSContrast(I); % Running this function, saving is value to C
fprintf('The value of RMS is %f\n', C); % Outputting the contrast of this image
J = Grayscale(I); % Running this function converts images to grayscale
imshow(J, 'Parent',app.UIAxes2) % Send the image attached to variable J to the GUI
end
% Button pushed function: GrayscaleImage2Button
function GrayscaleImage2ButtonPushed(app, ~)
global I2;
global J2;
C = RMSContrast(I2);
fprintf('The value of RMS is %f\n', C);
J2 = Grayscale(I2);
imshow(J2, 'Parent',app.UIAxes5)
end
% Button pushed function: GrayscaleImage3Button
function GrayscaleImage3ButtonPushed(app, ~)
global I3;
global J3;
C = RMSContrast(I3);
fprintf('The value of RMS is %f\n', C);
J3 = Grayscale(I3);
imshow(J3, 'Parent',app.UIAxes8)
end
% Button pushed function: RESETButton
function RESETButtonPushed(app, ~)
cla(app.UIAxes, 'reset');
cla(app.UIAxes2, 'reset');
cla(app.UIAxes3, 'reset');
cla(app.UIAxes4, 'reset');
cla(app.UIAxes5, 'reset');
cla(app.UIAxes6, 'reset');
cla(app.UIAxes7, 'reset');
cla(app.UIAxes8, 'reset');
cla(app.UIAxes9, 'reset');
title(app.UIAxes,"");
title(app.UIAxes2,"");
```

```
title(app.UIAxes3,"");
title(app.UIAxes4,"");
title(app.UIAxes5,"");
title(app.UIAxes6,"");
title(app.UIAxes7,"");
title(app.UIAxes8,"");
title(app.UIAxes9,"");
end
% Button pushed function: HistogramButton
function HistogramButtonPushed(app, ~)
global I;
global J;
Intensity=Hist(J); % Uses the histogram function to make the histogram for image 1
bar(app.UIAxes3,0:255,Intensity); % Updates the GUI for the histogram
end
% Button pushed function: HistogramButton2
function Histogram2ButtonPushed(app, ~)
global I2;
global J2;
Intensity=Hist(J2); % Uses the histogram function to make the histogram for image
2
bar(app.UIAxes6,0:255,Intensity); % Updates the GUI for the histogram
end
% Button pushed function: HistogramButton3
function Histogram3ButtonPushed(app, ~)
global J3;
Intensity=Hist(J3); % Uses the histogram function to make the histogram for image
3
bar(app.UIAxes9,0:255,Intensity); % Updates the GUI for the histogram
end
function ProcessedImageButtonPushed(app, ~)
global J;
MO = Processing(J); % Runs all the processing methods for image 1
figure() % Makes a figure of the processed image, will be generated when a button
is clicked
imshow(MO);
title ("Processed Image");
end
function ProcessedImage2ButtonPushed(app, ~)
global J2;
MO = Processing(J2); % Runs all the processing methods for image 2
figure() % Makes a figure of the processed image, will be generated when a button
is clicked
imshow(MO);
title ("Processed Image");
end
function ProcessedImage3ButtonPushed(app, ~)
global J3;
MO = Processing(J3); % Runs all the processing methods for image 3
figure() % Makes a figure of the processed image, will be generated when a button
is clicked
```

```
imshow(MO);
title ("Processed Image");
end
end
% Component initialization
methods (Access = private)
% Create UIFigure and components
function createComponents(app)
% Create UIFigure and hide until all components are created
app.UIFigure = uifigure('Visible', 'off');
app.UIFigure.Position = [100 100 960 600];
app.UIFigure.Name = 'GUI of License Plate Detection';
% Create UIAxes
app.UIAxes = uiaxes(app.UIFigure);
title(app.UIAxes, 'Image 1')
app.UIAxes.XTick = [];
app.UIAxes.YTick = [];
app.UIAxes.Color = [1 1 0];
app.UIAxes.Box = 'on';
app.UIAxes.ButtonDownFcn = createCallbackFcn(app, @UIAxesButtonDown, true);
colormap(app.UIAxes, 'summer')
app.UIAxes.Position = [20 400 280 150];
% Create UIAxes2
app.UIAxes2 = uiaxes(app.UIFigure);
title(app.UIAxes2, 'Grayscale Image 1')
app.UIAxes2.XTick = [];
app.UIAxes2.YTick = [];
app.UIAxes2.Color = [0.0588 1 1];
app.UIAxes2.Box = 'on';
app.UIAxes2.Position = [320 400 280 150];
% Create UIAxes3
app.UIAxes3 = uiaxes(app.UIFigure);
title(app.UIAxes3, 'Histogram of Image 1')
xlabel(app.UIAxes3, 'Intensity(Brightness)')
ylabel(app.UIAxes3, 'No. Of Pixels')
zlabel(app.UIAxes3, 'Z')
app.UIAxes3.Position = [620 400 280 150];
% Create UIAxes4
app.UIAxes4 = uiaxes(app.UIFigure);
title(app.UIAxes4, 'Image 2')
app.UIAxes4.XTick = [];
app.UIAxes4.YTick = [];
app.UIAxes4.Color = [1 1 0];
app.UIAxes4.Box = 'on';
app.UIAxes4.ButtonDownFcn = createCallbackFcn(app, @UIAxesButtonDown, true);
colormap(app.UIAxes4, 'summer')
app.UIAxes4.Position = [20 220 280 150];
% Create UIAxes5
app.UIAxes5 = uiaxes(app.UIFigure);
title(app.UIAxes5, 'Grayscale of Image 2')
app.UIAxes5.XTick = [];
```

```
app.UIAxes5.YTick = [];
app.UIAxes5.Color = [0.0588 1 1];
app.UIAxes5.Box = 'on';
app.UIAxes5.Position = [320 220 280 150];
% Create UIAxes6
app.UIAxes6 = uiaxes(app.UIFigure);
title(app.UIAxes6, 'Histogram of Image 2')
xlabel(app.UIAxes6, 'Intensity(Brightness)')
ylabel(app.UIAxes6, 'No. Of Pixels')
zlabel(app.UIAxes6, 'Z')
app.UIAxes6.Position = [620 220 280 150];
% Create UIAxes7
app.UIAxes7 = uiaxes(app.UIFigure);
title(app.UIAxes7, 'Image 3')
app.UIAxes7.XTick = [];
app.UIAxes7.YTick = [];
app.UIAxes7.Color = [1 1 0];
app.UIAxes7.Box = 'on';
app.UIAxes7.ButtonDownFcn = createCallbackFcn(app, @UIAxesButtonDown, true);
colormap(app.UIAxes7, 'summer')
app.UIAxes7.Position = [20 40 280 150];
% Create UIAxes8
app.UIAxes8 = uiaxes(app.UIFigure);
title(app.UIAxes8, 'Grayscale of Image 3')
app.UIAxes8.XTick = [];
app.UIAxes8.YTick = [];
app.UIAxes8.Color = [0.0588 1 1];
app.UIAxes8.Box = 'on';
app.UIAxes8.Position = [320 40 280 150];
% Create UIAxes9
app.UIAxes9 = uiaxes(app.UIFigure);
title(app.UIAxes9, 'Histogram of Image 3')
xlabel(app.UIAxes9, 'Intensity(Brightness)')
ylabel(app.UIAxes9, 'No. Of Pixels')
zlabel(app.UIAxes9, 'Z')
app.UIAxes9.Position = [620 40 280 150];
% Create LoadImageButton
app.LoadImageButton = uibutton(app.UIFigure, 'push'); % connecting the button to
the GUI
app.LoadImageButton.ButtonPushedFcn = createCallbackFcn(app,
@LoadImageButtonPushed, true); % causes the function to run when the button is
pressed
app.LoadImageButton.Position = [110 380 100 20]; % where the button is located on
the GUI
app.LoadImageButton.Text = 'Load Image 1'; % the label attached to the button
% Create LoadImage2Button
app.LoadImage2Button = uibutton(app.UIFigure, 'push');
app.LoadImage2Button.ButtonPushedFcn = createCallbackFcn(app,
@LoadImage2ButtonPushed, true);
app.LoadImage2Button.Position = [110 200 100 20];
app.LoadImage2Button.Text = 'Load Image 2';
```

```
% Create LoadImage3Button
app.LoadImage3Button = uibutton(app.UIFigure, 'push');
app.LoadImage3Button.ButtonPushedFcn = createCallbackFcn(app,
@LoadImage3ButtonPushed, true);
app.LoadImage3Button.Position = [110 20 100 20];
app.LoadImage3Button.Text = 'Load Image 3';
% Create GrayscaleImageButton
app.GrayscaleImageButton = uibutton(app.UIFigure,'push');
app.GrayscaleImageButton.ButtonPushedFcn = createCallbackFcn(app,
@GrayscaleImageButtonPushed, true);
app.GrayscaleImageButton.Position = [410 380 100 20];
app.GrayscaleImageButton.Text = 'Grayscale Image';
% Create GrayscaleImage2Button
app.GrayscaleImage2Button = uibutton(app.UIFigure,'push');
app.GrayscaleImage2Button.ButtonPushedFcn = createCallbackFcn(app,
@GrayscaleImage2ButtonPushed, true);
app.GrayscaleImage2Button.Position = [410 200 100 20];
app.GrayscaleImage2Button.Text = 'Grayscale Image 2';
% Create GrayscaleImage3Button
app.GrayscaleImage3Button = uibutton(app.UIFigure,'push');
app.GrayscaleImage3Button.ButtonPushedFcn = createCallbackFcn(app,
@GrayscaleImage3ButtonPushed, true);
app.GrayscaleImage3Button.Position = [410 20 100 20];
app.GrayscaleImage3Button.Text = 'Grayscale Image';
% Create ProcessedImageButton
app.ProcessedImageButton = uibutton(app.UIFigure,'push');
app.ProcessedImageButton.ButtonPushedFcn = createCallbackFcn(app,
@ProcessedImageButtonPushed, true);
app.ProcessedImageButton.Position = [200 560 180 25];
app.ProcessedImageButton.Text = 'Processed Version of Image 1';
% Create ProcessedImage2Button
app.ProcessedImage2Button = uibutton(app.UIFigure,'push');
app.ProcessedImage2Button.ButtonPushedFcn = createCallbackFcn(app,
@ProcessedImage2ButtonPushed, true);
app.ProcessedImage2Button.Position = [380 560 180 25];
app.ProcessedImage2Button.Text = 'Processed Version of Image 2';
% Create ProcessedImage3Button
app.ProcessedImage3Button = uibutton(app.UIFigure,'push');
app.ProcessedImage3Button.ButtonPushedFcn = createCallbackFcn(app,
@ProcessedImage3ButtonPushed, true);
app.ProcessedImage3Button.Position = [560 560 180 25];
app.ProcessedImage3Button.Text = 'Processed Version of Image 3';
% Create RESETButton
app.RESETButton = uibutton(app.UIFigure, 'push');
app.RESETButton.ButtonPushedFcn = createCallbackFcn(app, @RESETButtonPushed,
true);
app.RESETButton.WordWrap = 'on';
app.RESETButton.BackgroundColor = [1 0 0];
app.RESETButton.FontSize = 14;
app.RESETButton.FontWeight = 'bold';
app.RESETButton.Position = [20 560 100 25];
```

```
app.RESETButton.Text = 'RESET';
% Create HistogramButton
app.HistogramButton = uibutton(app.UIFigure, 'push');
app.HistogramButton.ButtonPushedFcn = createCallbackFcn(app,
@HistogramButtonPushed, true);
app.HistogramButton.Position = [710 380 130 20];
app.HistogramButton.Text = 'Histogram of Image 1';
% Create Histogram2Button
app.Histogram2Button = uibutton(app.UIFigure, 'push');
app.Histogram2Button.ButtonPushedFcn = createCallbackFcn(app,
@Histogram2ButtonPushed, true);
app.Histogram2Button.Position = [710 200 130 20];
app.Histogram2Button.Text = 'Histogram of Image 2';
% Create Histogram3Button
app.Histogram3Button = uibutton(app.UIFigure, 'push');
app.Histogram3Button.ButtonPushedFcn = createCallbackFcn(app,
@Histogram3ButtonPushed, true);
app.Histogram3Button.Position = [710 20 130 20];
app.Histogram3Button.Text = 'Histogram of Image 3';
% Show the figure after all components are created
app.UIFigure.Visible = 'on';
end
end
% App creation and deletion
methods (Access = public)
% Construct app
function app = Final
% Create UIFigure and components
createComponents(app)
% Register the app with App Designer
registerApp(app, app.UIFigure)
end
% Code that executes before app deletion
function delete(app)
% Delete UIFigure when app is deleted
delete(app.UIFigure)
end
end
end
function C = RMSContrast(I) %function to work out contrast of images
[M, N] = size(I); % size of the image (no. of rows and columns)
me = mean(I(:)); % average brightness of image
C = sqrt(sum((I(:) - me).^2)/(M*N)); % RMS Equation
end
function Gray = Grayscale(I)
% Extract channels
R = I(:,:,1);
G = I(:,:,2);
B = I(:,:,3);
% The normal values of rgb2gray
WR = 0.299;
```

```
WG = 0.587;
WB = 0.114;
% Grayscale formula
Gray = WR*R + WG*G + WB*B;
end
function MO = Processing(J)
J=medfilt2(J,[3,3]); % Median filter
SE = strel('rectangle', [3 15]); % Structuring element
b = imbinarize(J); % Binarization
IO = imopen(b,SE); % Opening
MO = imclose (IO,SE); % Closing
end
function H = Hist(I)
[rows, cols] = size(I); % Finds how many pixels the image has
H = zeros(1, 256); % Creates empty area of 256 zeros
% For loops to iterate through everything
for r = 1:rows % Starts at first row
for c = 1:cols % Starts at first column
Var = I(r, c);
H(Var + 1) = H(Var + 1) + 1;
% The x-axis is the intensity(brightness) of the pixel
% The y-axis is the no. of pixels that share the same shade of grey
% If the left side has more lines the image is closer to black,
% If the right has more the image is closer to white
end
end
end
```

Figure of Tables